

INTEGRATED PROJECT SYNOPSIS

On

HealthSphere

SUBMITTED

TO

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FOR

INTEGRATED PROJECT (22CS038)

Submitted By:

Prachi Kapoor, 2310990852
Madhav Chuttani, 2310991336
Utkarsh, 2310991233

Submitted To:

Dr. Ravi Kumar Sachdeva

Index

Sr No.	Topic	Page No.
1	Problem Statement	3-4
2	Objective & Key Learning's	5
3	Functional Requirements and Non-Functional Requirements	6-7
4	High Level and Low Level Design	8-21
5	Advantages/ Disadvantages	22
6	Conclusion and Future Improvements: Potential Bottlenecks	23
7	References	24

1. Problem Statement

With the rapid advancement of digital healthcare technologies, users increasingly rely on online platforms for medical information, symptom checking, and consultation. However, most existing healthcare applications either provide generic information or require direct consultation with doctors, making the process time-consuming, expensive, and sometimes inaccessible.

Studies indicate that while users actively search for health-related information online, they often face challenges such as inaccurate self-diagnosis, lack of personalized recommendations, and delayed medical assistance. Additionally, current platforms fail to integrate real-time interaction with healthcare professionals in an efficient and structured manner.

In daily life, individuals frequently require:

- Quick analysis of symptoms
- Preliminary guidance on possible health conditions
- Basic medicine suggestions for common illnesses
- Immediate access to medical consultation when needed

However, there exists a significant gap:

Lack of an intelligent, integrated platform that combines AI-based symptom analysis with real-time doctor consultation in a seamless and accessible manner.

Current solutions such as health websites and telemedicine apps are:

- **Generic** (non-personalized responses)
- **Time-consuming** (manual appointment processes)
- **Limited in real-time interaction**
- **Lacking affordability and instant accessibility**

Problem Definition:

To develop an AI-powered healthcare platform that enables users to:

- Input symptoms and receive intelligent, AI-driven analysis
- Get basic medicine suggestions for common conditions
- Connect with certified doctors through real-time chat (WebSocket-based)
- Access paid consultation services via secure payment integration (Razorpay)

2. Objectives & Key Learnings

2.1. Objectives:

- To design and develop an AI-powered healthcare platform for quick and accessible medical assistance
 - To enable users to input symptoms and receive intelligent health analysis using AI integration
 - To provide basic medicine suggestions for common health conditions
 - To facilitate real-time doctor consultation through WebSocket-based chat system
 - To integrate secure online payment (Razorpay) for paid doctor consultations
 - To create a reliable and user-friendly healthcare ecosystem for timely guidance and support
-

2.2. Key Learnings:

- Understanding client-server architecture in healthcare-based applications
- Implementation of REST APIs for handling user data and medical queries
- Learning real-time communication using WebSockets for doctor-patient interaction
- Working with backend frameworks like Node.js / NestJS for scalable system design
- Database modeling for user records, symptom data, and consultation history
- Integrating third-party services (AI APIs & Razorpay payment gateway)
- Designing low-latency and responsive systems for real-time healthcare support

3. Functional and Non-Functional Requirements

3.1. Functional Requirements:

1. User Authentication

- Users can register and login using email/mobile authentication
- **Secure login with proper validation**

2. User Profile Management

- Users can manage basic details: name, age, gender, medical history (optional)
- Store past consultation and symptom records

3. Symptom Input & AI Analysis

- Users can enter symptoms through a structured interface
- AI processes input and provides:
 - Possible health conditions
 - Basic guidance

4. Medicine Suggestion System

- Provide basic medicine recommendations for common illnesses
- Include dosage guidelines (general only) and precautions

5. Doctor Consultation (Real-Time Chat)

- Users can connect with doctors via WebSocket-based real-time chat
- Doctors can view user symptoms and provide advice

6. Payment Integration

- Enable secure payments using Razorpay for paid consultations
- Access to chat unlocked after successful payment

7. Consultation History

- Store previous chats, prescriptions, and symptom analysis
- Users can review past medical interactions

3.2. Non-Functional Requirements:

- **Scalability:** System should handle a growing number of users and consultations
- **Performance:** Fast AI response and low-latency real-time chat
- **Security:** Secure user data, encrypted communication, and safe payment processing
- **Reliability:** System should ensure minimal downtime and stable performance
- **Usability:** Simple, clean, and intuitive UI for all users
- **Privacy:** User medical data should be confidential and access-controlled

4. High Level Design and Low Level Design

4.1 High Level Design (HLD)

4.1.1. Overview

High-Level Design (HLD) provides a macro-level view of the system architecture, describing how different components interact with each other. It focuses on system structure, data flow, and major modules without going into implementation details.

The proposed system follows a client-server architecture, where the frontend communicates with the backend via REST APIs, while real-time communication (doctor consultation) is handled using Web Sockets. The backend manages business logic, AI integration, and data storage.

4.1.2. System Architecture

The system consists of four primary layers:

a) Presentation Layer (Frontend)

- Built using React
- Handles user interface, symptom input, and chat interface
- Sends API requests and receives responses

b) Application Layer (Backend)

- Built using Node.js and Express / NestJS
- Responsible for:
 - Business logic
 - Authentication
 - AI integration for symptom analysis
 - Request processing
 - WebSocket communication for real-time chat
 - Interaction with database and external services

c) Data Layer (Database)

- Stores all system data including:
 - Users
 - Medical history

- Symptoms and AI analysis results
- Consultation records
- Chat messages
- Payments

d) External Services

- AI APIs for symptom analysis and recommendations
 - Razorpay for payment processing
 - OTP/SMS/Email services for user verification and notifications
-

4.1.3. Data Flow

The system follows a request-response + real-time communication flow:

1. User enters symptoms or initiates consultation via frontend
 2. Frontend sends request to backend via REST API
 3. Backend processes request and interacts with AI service
 4. Backend stores/retrieves data from database
 5. Response is sent back to frontend
 6. For doctor consultation, WebSocket connection is established for real-time chat
-

4.1.4. Major System Components

a) Authentication Module

Handles secure login/signup using email/mobile with OTP verification

b) User Management Module

Manages user profiles, medical history, and consultation records

c) Symptom Analysis Module (AI Integration)

Processes user symptoms and provides possible conditions and guidance

d) Medicine Suggestion Module

Provides basic medicine recommendations for common illnesses

e) Doctor Consultation Module

- Connects users with doctors

- Enables real-time chat using WebSockets

f) Payment Module

Handles secure payments using Razorpay before consultation

g) Chat Module

Manages real-time messaging between doctor and patient

h) History & Records Module

Stores previous consultations, prescriptions, and reports

4.1.5. Architectural Style

The system follows:

a) Client-Server Architecture

Clear separation between frontend and backend

b) Layered Architecture

- Presentation Layer
- Application Layer
- Data Layer

c) Service-Oriented Design

Each functionality (AI, payment, chat) is handled by independent services

4.1.6. Security Considerations

- Secure authentication using OTP/email verification
 - Encrypted communication between client and server
 - Secure handling of medical data and user privacy
 - Role-based access (user, doctor)
 - Secure payment processing via Razorpay
-

4.1.7. Scalability Considerations

- Modular architecture allows independent scaling of services
 - WebSocket server can be scaled for multiple concurrent chats
 - Backend can be extended using NestJS / microservices
 - Database can be optimized or scaled (e.g., MongoDB Atlas)
 - External services (AI, payment) can be replaced or upgraded
-

4.1.8. Limitations (MVP)

- AI may provide basic guidance only (not 100% accurate diagnosis)
 - No video/audio consultation (chat-based only)
 - Limited dataset for AI recommendations
 - Dependency on third-party APIs
-

4.1.9. Future Enhancements

- Video consultation with doctors
 - Advanced AI model for accurate diagnosis
 - Mobile application (Flutter / React Native)
 - Integration with wearable health devices
 - Multi-language support
 - E-prescription and report download
-

4.1.10. Conclusion

The High-Level Design presents a structured architectural blueprint of the AI-powered healthcare system. It ensures modularity, scalability, and efficient real-time communication, forming a strong foundation for development and future enhancements.

4.2 LOW LEVEL DESIGN (LLD)

4.2.1 Overview

The Low-Level Design (LLD) describes the internal structure of the system, including classes, data models, relationships, and logic implementation. The system follows a modular, service-oriented architecture, where each component is responsible for a specific functionality.

The design is centered around a user-health interaction model, where users input symptoms, receive AI-based analysis, and connect with doctors for consultation.

4.2.2 Design Approach

The system is designed using the following principles:

- **Modular Design**

Each feature is divided into independent modules such as:

- Authentication
- Symptom Analysis (AI Integration)
- Medicine Suggestion
- Doctor Consultation
- Payment System
- Chat System

This ensures maintainability and scalability.

- **Separation of Concerns**

- Controllers → Handle API requests
 - Services → Handle business logic (AI processing, chat handling, payments)
 - Database Layer → Handles data storage
-

- **Health-Centric Architecture**

The system revolves around health queries (symptoms) as the core entity, enabling:

- AI-based analysis
 - Doctor consultation
 - Medical guidance
-

4.2.3 Core Entities Description

a) User

Represents a person using the platform

Stores personal details, login credentials, and medical history

b) Symptom Record

Stores symptoms entered by the user

Acts as input for AI-based analysis

c) Analysis Result

Stores AI-generated results

Includes possible conditions and basic recommendations

d) Medicine Suggestion

Provides suggested medicines for common conditions

Includes general dosage and precautions

e) Doctor

Represents verified medical professionals

Stores specialization, availability, and consultation details

f) Consultation

Represents a session between user and doctor

Acts as a container for chat and medical interaction

g) Chat Message

Stores real-time messages between doctor and user

Ensures context-based communication

h) Payment

Handles transaction details for paid consultations

Stores payment status and transaction ID

i) Medical History

Stores past consultations, symptoms, and prescriptions

Helps in future reference and better analysis

4.2.4 System Workflow

The system follows this flow:

1. User registers and logs in
 2. User enters symptoms
 3. AI analyzes symptoms and provides results
 4. User views suggestions and decides next step
 5. User initiates doctor consultation
 6. Payment is completed via Razorpay
 7. Real-time chat (WebSocket) is enabled
 8. Consultation is completed
 9. Data is stored in medical history
-

4.2.5 Access Control Mechanism

The system enforces access restrictions:

- Only authenticated users can access features
 - Only paid users can start doctor consultation
 - Only assigned doctor and user can access chat
 - Medical data is private and access-controlled
-

4.2.6 Service Layer Design

Each functionality is handled by a dedicated service:

- AuthService → authentication & verification
- SymptomService → symptom input & processing
- AIService → AI-based analysis

- ConsultationService → doctor session management
 - ChatService → real-time messaging (WebSockets)
 - PaymentService → Razorpay integration
 - HistoryService → medical records storage
-

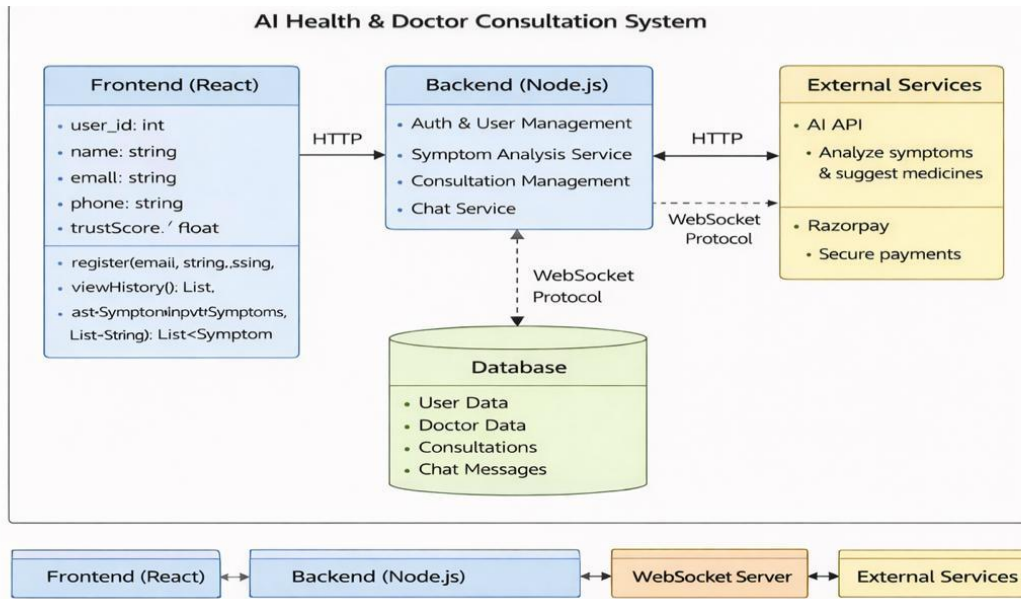
4.2.7 Design Constraints

- AI provides basic guidance only (not a replacement for doctors)
 - No video/audio consultation (chat-based MVP)
 - Dependency on third-party AI APIs
 - Limited personalization in early version
-

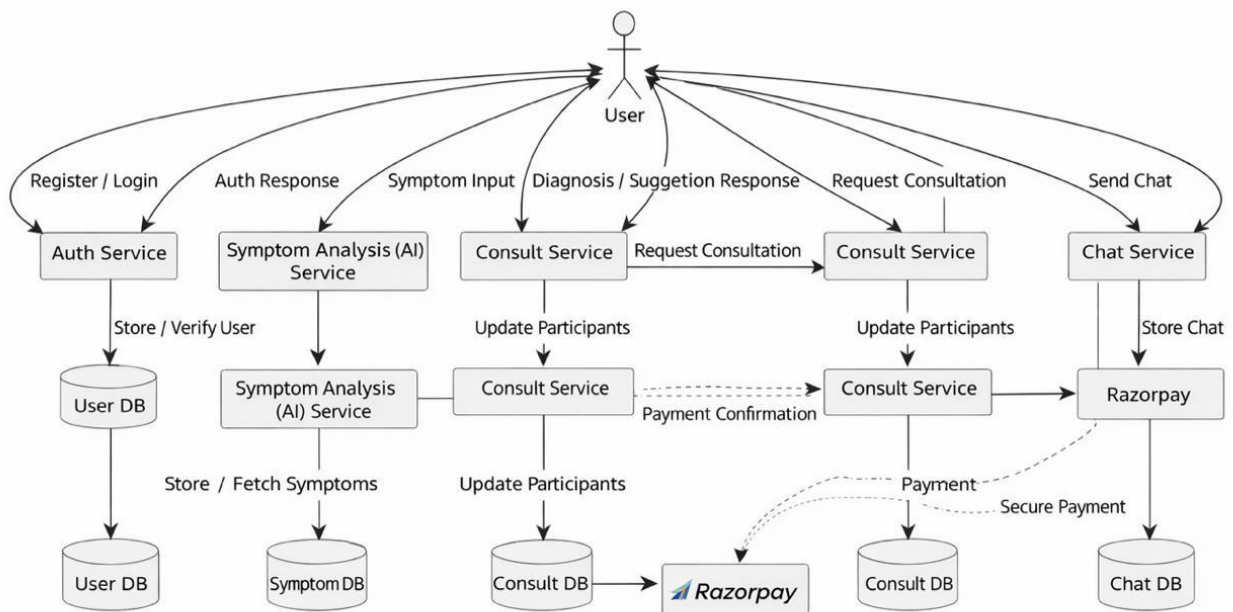
4.2.8 Key Design Decisions

- AI-first approach for quick symptom analysis
- On-demand doctor consultation instead of continuous availability
- Real-time chat using WebSockets for instant communication
- Secure payment before consultation to ensure commitment
- Medical history tracking for better user experience

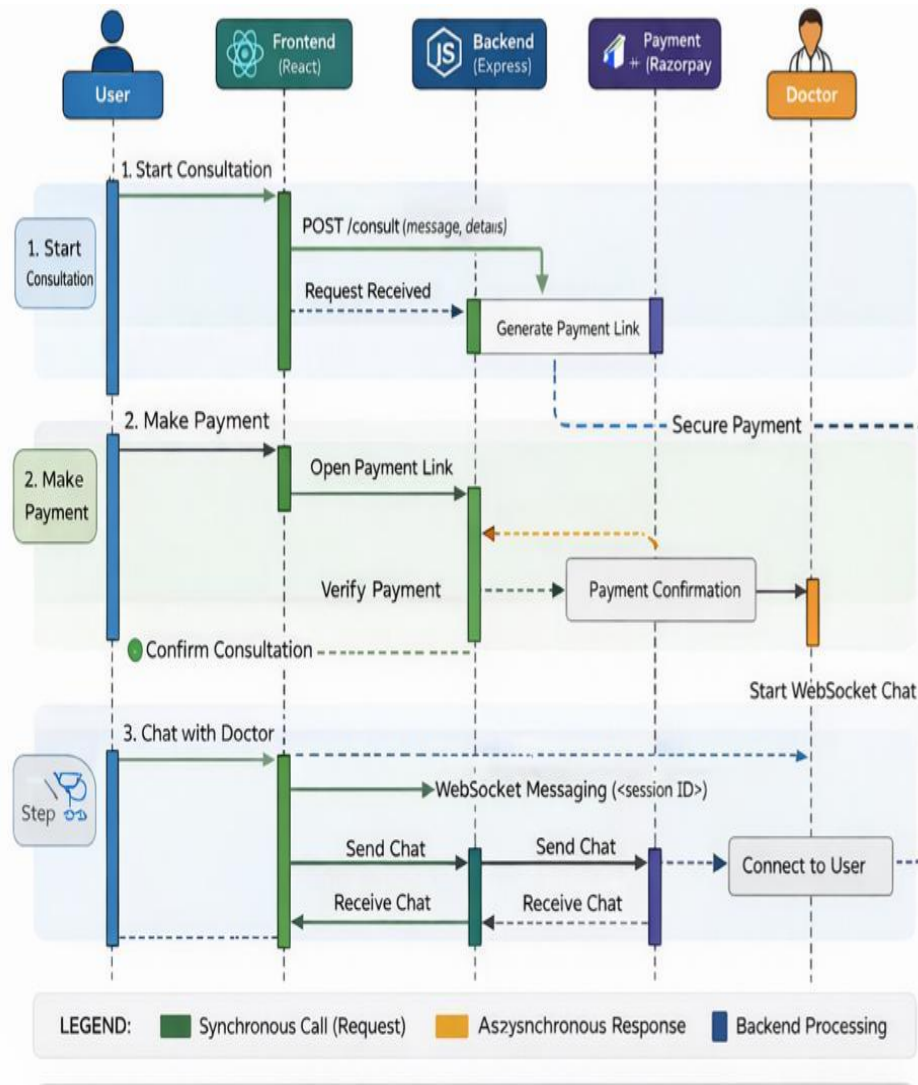
4.1.9 Diagrams



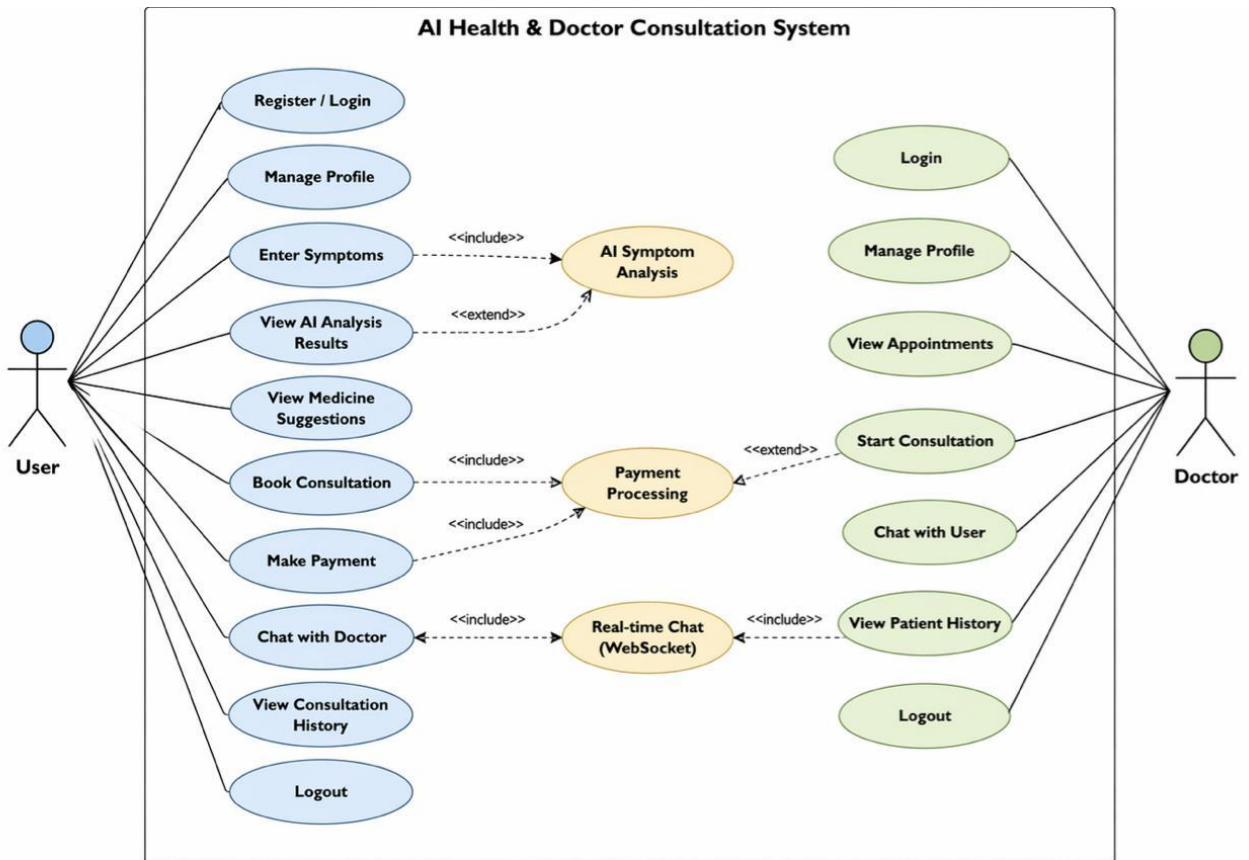
HLD Diagram



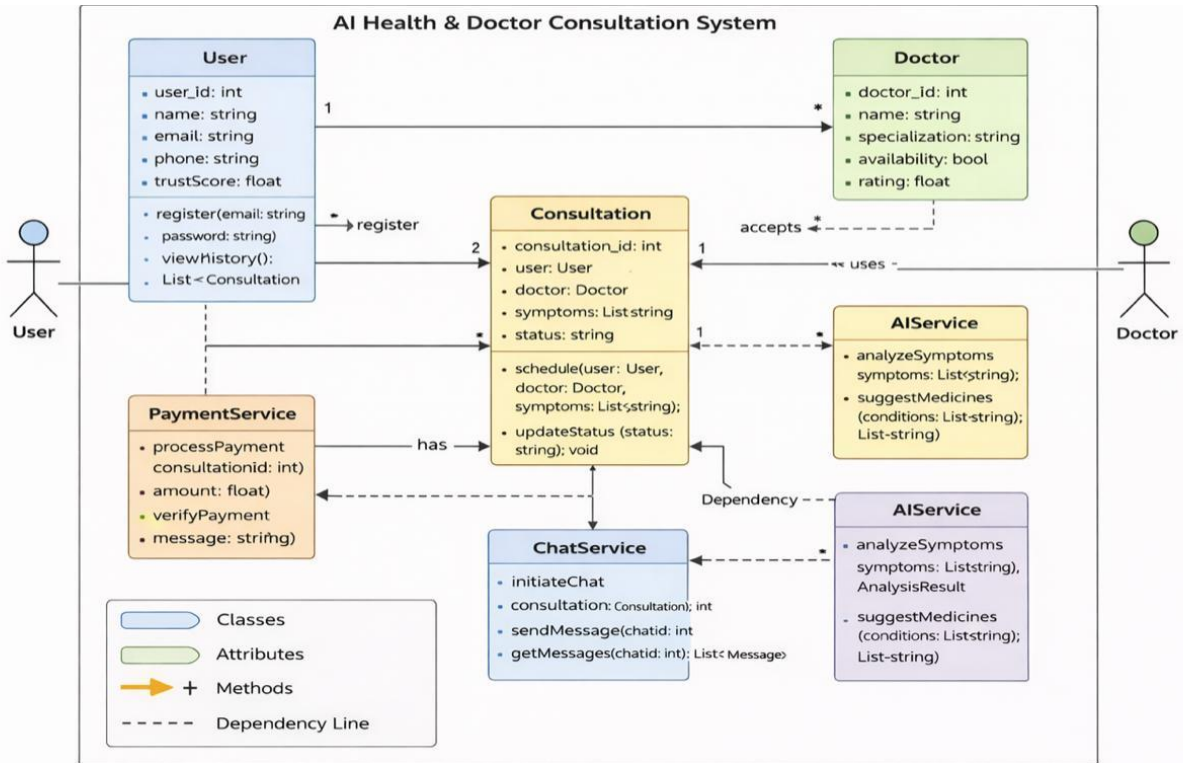
Data Flow



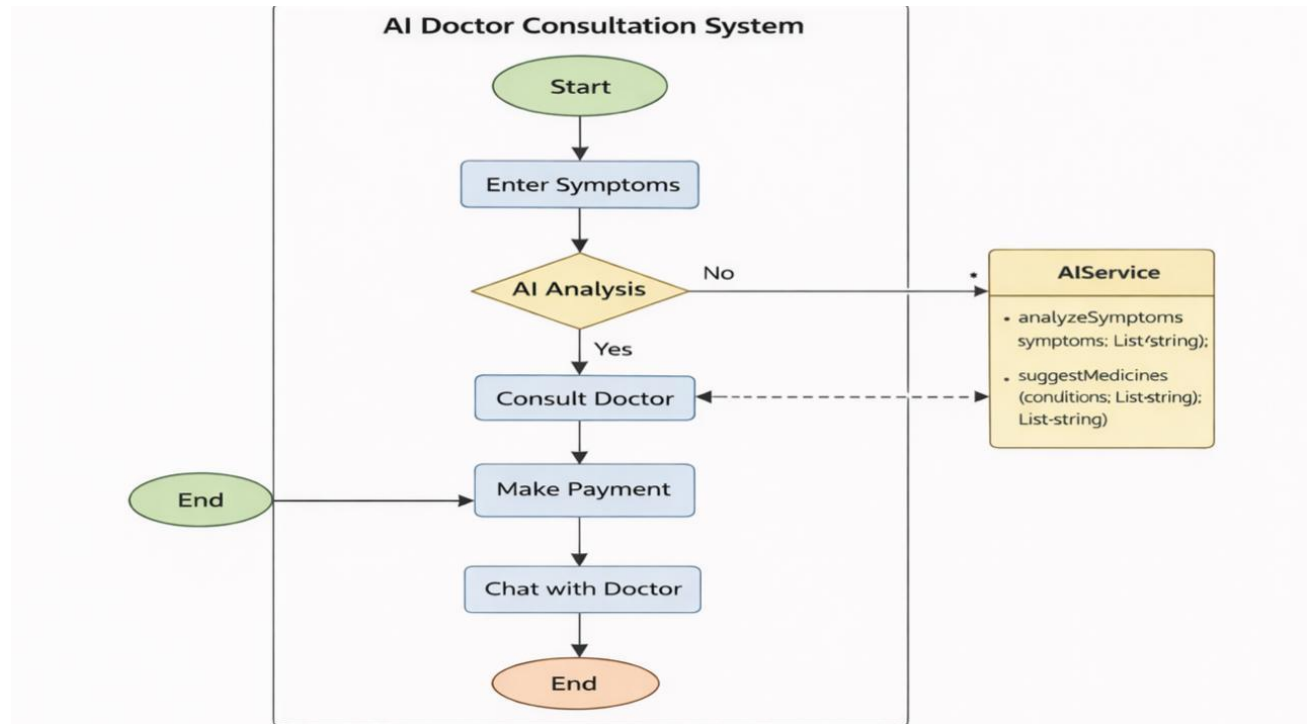
Sequence Diagram



USE Case Diagram



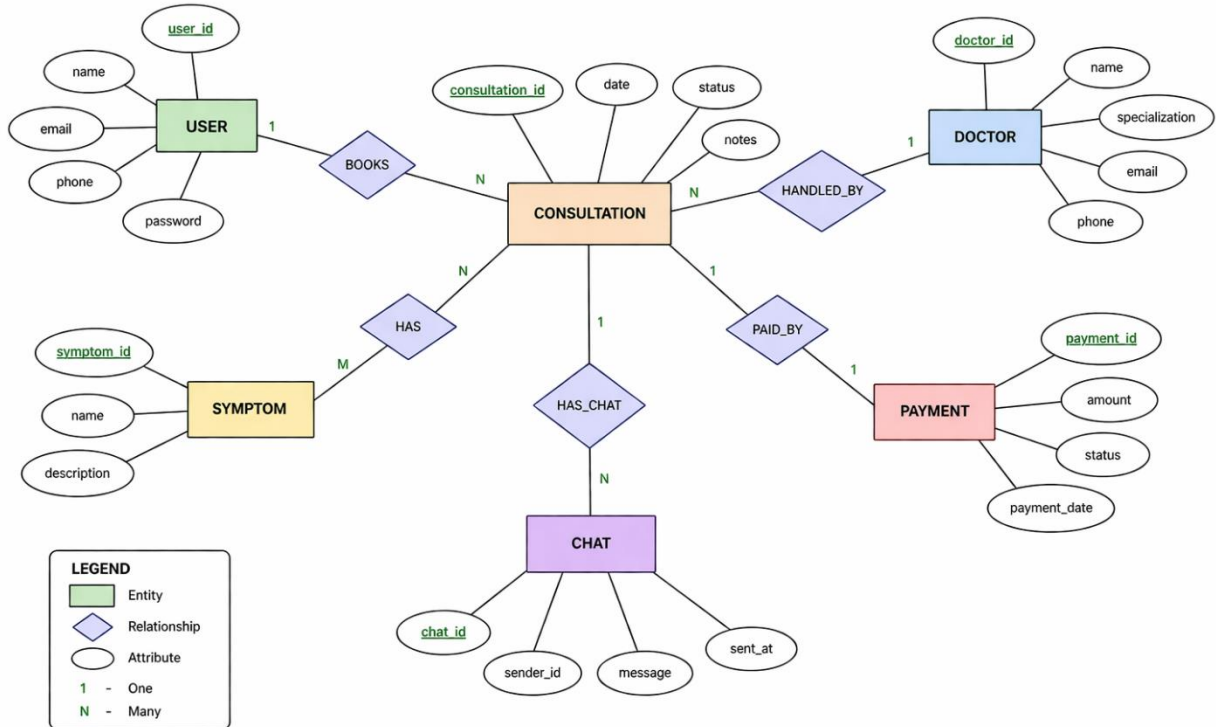
Class Diagram



Activity Diagram



ER Diagram – AI Health & Doctor Consultation System



ER Diagram

5. Advantages and Disadvantages

Advantages:

- Provides instant AI-based symptom analysis, saving time for users
 - Enables quick preliminary diagnosis without immediate doctor visit
 - Offers 24/7 accessibility for basic healthcare guidance
 - Facilitates real-time doctor consultation through chat
 - Reduces unnecessary hospital visits for minor issues
 - Maintains digital medical history for future reference
 - Integrates secure online payment (Razorpay) for seamless consultations
-

Disadvantages:

- AI suggestions may not always be 100% accurate
- Cannot fully replace professional medical diagnosis
- Requires internet connectivity for access
- Dependency on third-party services (AI APIs, payment gateway)
- Potential concerns regarding data privacy and security

6. Conclusion and Future Improvements: Potential Bottlenecks

Conclusion:

The proposed system aims to enhance digital healthcare by integrating AI-based symptom analysis with real-time doctor consultation. It provides users with quick medical guidance, reduces dependency on physical visits for minor issues, and ensures timely access to healthcare services.

By combining AI intelligence, secure payments, and real-time communication, the platform creates an efficient, accessible, and user-friendly healthcare ecosystem.

Potential Bottlenecks:

- AI Accuracy: AI-generated suggestions may not always be fully reliable
- Doctor Availability: Real-time consultation depends on doctor presence
- Scalability Issues: Handling large number of users and concurrent chats
- Data Privacy & Security: Protecting sensitive medical information
- Third-party Dependency: Reliance on AI APIs and Razorpay services

Future Improvements:

- Implementation of advanced AI models for more accurate diagnosis
- Introduction of video/audio consultation with doctors
- Integration with wearable health devices (smartwatch, fitness trackers)
- Multi-language support for wider accessibility
- Development of mobile application (Flutter / React Native)

7. References

i. Client–Server Architecture - System Design, GeeksforGeeks

<https://www.geeksforgeeks.org/system-design/client-server-architecture-system-design/>

ii. Node.js Runtime Environment, Wikipedia

<https://en.wikipedia.org/wiki/Node.js>

iii. Express.js Framework Overview, Wikipedia

<https://en.wikipedia.org/wiki/Express.js>

iv. React.js Documentation, Meta

<https://react.dev/>

v. PlantUML – UML Diagram Tool

<https://plantuml.com/>

vi. Postman – API Testing Tool

<https://www.postman.com/>